



ACOUSTIC MACHINE FAULT DETECTION SYSTEM



A PROJECT REPORT

Submitted by

KIRUTHIKA NATARAJAN (22IT018)

MUTHUKRISHNAN P (22IT023)

VARUNA PRIYA S (22IT052)

In partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

INFORMATION TECHNOLOGY

NANDHA ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

ERODE – 638 052

MAY 2026

NANDHA ENGINEERING COLLEGE
(An Autonomous Institution, Affiliated to Anna University, Chennai)
ERODE - 638 052

BONAFIDE CERTIFICATE

This is to certify that the project report entitled “**ACOUSTIC MACHINE FAULT DETECTION SYSTEM**” is the bonafide record of project work done by **KIRUTHIKA NATARAJAN (22IT018), MUTHUKRISHNAN P (22IT023), VARUNA PRIYA S (22IT052)** in partial fulfillment of the requirements for the award of the Degree of Bachelor of Technology in INFORMATION TECHNOLOGY of Anna University, Chennai during the year 2025-26.

SUPERVISOR

Mr. R. RAGUNATH, M.E.

Assistant Professor,
Department of IT,
Nandha Engineering College,
Erode 638 052.

HEAD OF THE DEPARTMENT

Dr. G. R. SREEKANTH, M.E, Ph.D

Professor and Head of Department,
Department of IT,
Nandha Engineering College,
Erode 638 052.

Submitted for the End Semester 22ITD01- Project Work Viva-Voce Examination
held on

Internal Examiner

External Examiner

ACKNOWLEDGEMENT

We first and foremost, our heartfelt thanks to our **Parents** for giving us an opportunity to do this engineering course successfully.

We wish to express profound gratitude to **Thiru. V. Shanmugan, Chairman**, Sri Nandha Educational Trust, **Thiru. S. Nandhakumar Pradeep, Joint Managing Trustee**, Sri Nandha Educational Trust and **Thiru. S. Thirumoorthi, Joint Managing Trustee**, Sri Nandha Educational Trust for providing opportunities in all possible ways for our improvement.

We wish to convey our gratefulness to our beloved **Principal, Dr. U. S. Ragupathy, Ph.D.** for his strong support and motivation towards a great level of success in our career.

We take this opportunity to express our thanks to our beloved **Head of the Department, Dr. G. R. Sreekanth, Ph.D** for his help and encouragement.

We articulate our genuine thanks to our **Project Co-ordinator, Mr. R. Ragunath, Assistant Professor** who have been the key spring of motivation to us throughout the completion of our course and project work.

We sincerely thank our **Supervisor, Mr. R. Ragunath, M.E, Assistant Professor**, Department of **Information Technology** for his valuable guidance to complete this project.

We express our sincere thanks to all **Information Technology** Department Faculty Members for their help in completing this project.

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
	ABSTRACT	
	LIST OF FIGURES	
	LIST OF ABBREVIATIONS	
1.	INTRODUCTION	1
	1.1 NEED FOR THE STUDY	2
	1.2 OBJECTIVES FOR THE STUDY	3
2.	REVIEW OF LITERATURE	4
	2.1 TECHNOLOGICAL ACHIVEMENT	6
	2.2 IMPACT ON CUSTOMER EXPERIENCE & EFFICIENCY	8
3.	PROJECT LIFECYCLE	9
4.	WORKFLOW OF PATIENT DATA MODULE	12
5.	LANGUAGES USED	14
6.	HARDWARE AND SOFTWARE REQUIREMENTS	
	I. HARDWARE	16
	II. SOFTWARE	16
7.	MODULES	
	7.1 MODULE DESCRIPTIONS	
	7.1.1 WORKING PROCESS	18
	7.1.2 CONTROLS	22
8.	RESULTING CONFIGURATION AND OUTCOMES	25

9.	APPLICATIONS OF THE PROJECT	28
10.	SOURCE CODE	
10.1	APP.PY	30
10.2	AUDIO_PROCESSOR.PY	31
10.3	DATABASE.PY	32
10.4	NOTIFIER.PY	33
10.5	ML_MODEL.PY	35
10.6	TRAIN_DATASET.PY	36
10.7	GENERATE_APPLIANCE_DATA.PY	38
10.8	GENERATE_TEST_DATA.PY	39
11	PROJECT SCREENSHOTS	41
12	CONCLUSION	44
13	REFERENCES	45

ABSTRACT

Machines produce unique sound patterns during operation, and any unusual change in these sounds can indicate the presence of a fault. Detecting such issues at an early stage helps prevent major breakdowns, reduce maintenance costs, and improve overall machine performance. This project presents a software-based solution for machine fault detection using acoustic analysis and sensor data. The system processes machine sounds using audio analysis techniques and extracts key features such as frequency patterns and energy variations to understand the machine's condition. A simple classification approach is used to identify whether the machine is working normally or showing abnormal behavior. The results are displayed through a web-based dashboard, allowing users to easily monitor machine status, view analysis results, and track fault history. This solution is simple, cost-effective, and user-friendly, and in future, it can be enhanced by integrating IoT for real-time monitoring and remote access.

LIST OF FIGURES

S.NO	FIG.NO	DESCRIPTION	PAGE.NO
1.	4.1	work flow chart of Machine Fault Detection	12
2.	11.1	Dashboard Page	41
3.	11.2	Record & Detect Page	
4.	11.3	Anomaly History Log Page	42
5.	11.4	Recent Alerts Page	
6.	11.5	SMS Alert Notification	43

LIST OF ABBREVIATIONS

S.NO	SYMBOLS/ ABBREVIATIONS	EXPANSION
1.	AI	Artificial Intelligence
2.	ML	Machine Learning
3.	API	Application Programming Interface
4.	REST	Representational State Transfer
5.	HTTP	Hypertext Transfer Protocol
6.	JSON	Javascript Object Notation
7.	SMTP	Simple Mail Transfer Protocol
8.	SQL	Structured Query Language
9.	IOT	Internet of Things
10.	SMS	Short Message Service
11.	MFCC	Mel Frequency Cepstral Coefficients
12.	AWS	Amazon Web Service
13.	ERP	Enterprise Resource Planning
14.	CMMS	Computerized Maintenance Management System
15.	RAM	Random Access Memory
16.	KPI	Key Performance Indicator
17.	AJAX	Asynchronous Javascript and XML
18.	NPM	Node Package Manager
19.	SSL	Secure Socket Layer

1. INTRODUCTION

The Acoustic Machine Fault Detection System is a web-based solution developed to monitor machine health by analyzing the sounds produced during operation. Every machine generates a unique sound pattern, and any change in this pattern can indicate a potential fault. This system is designed to automatically detect such abnormalities using audio analysis and deep learning techniques, providing a simple and efficient way to ensure machine reliability.

The system offers a clean and user-friendly interface where users can easily record live audio or upload sound files for analysis. It provides real-time results along with visual representations such as waveform and spectrogram, helping users better understand the condition of the machine. The dashboard displays important information like total scans, healthy operations, and detected anomalies, making monitoring easy and effective.

To improve response time and prevent unexpected failures, the system includes an alert mechanism that sends notifications through SMS and email whenever a fault is detected. This helps users take immediate action and reduces machine downtime. The application is fully web-based, allowing access from any device without the need for complex installations.

With the growing need for smart and automated maintenance systems, traditional methods such as manual inspection are becoming less effective. They are time-consuming, prone to errors, and often fail to detect early-stage faults. This system provides a modern solution by automating the fault detection process using sound analysis, making it cost-effective and suitable for real-world applications.

The system consists of three distinct interfaces:

1. **Dashboard:** Provides an overview of total scans, healthy sounds, detected anomalies, and overall system activity.
2. **Record & Detect:** Allows users to record live audio or upload files to analyze machine conditions and detect faults instantly.
3. **History Log:** Stores all previous detection results with details such as date, device type, and status.
4. **Alerts System:** Displays recent fault alerts and sends automatic SMS and Email notifications when anomalies are detected.

1.1 Need for the Study

Traditional machine monitoring systems mainly depend on manual inspection and expensive hardware sensors, which can lead to several issues such as:

- Delay in identifying machine faults
- Increased maintenance cost and downtime
- Difficulty in continuous monitoring
- Dependency on skilled professionals
- Lack of instant alert systems
- Poor tracking of machine performance history
- Limited visualization of machine condition

The System addresses these challenges through:

- Automatic fault detection using sound analysis
- SMS and Email alert system for quick response
- Centralized web dashboard for easy access
- Storage and tracking of historical data

1.2 Objectives of the Study

The main objectives of developing the Acoustic Machine Fault Detection System are:

- To develop a web-based platform for monitoring machine health
- To detect faults automatically using sound-based analysis
- To provide real-time audio recording and file upload features.
- To visualize audio signals using waveform and spectrogram
- To implement SMS and Email alerts for fault detection.
- To maintain a history log of machine conditions
- To reduce machine downtime through early detection
- To design a system that can be extended with IoT in the future

2. REVIEW OF LITERATURE

ACOUSTIC-BASED MACHINE FAULT DETECTION

Acoustic-based machine fault detection is a modern approach that uses sound signals generated during machine operation to identify faults. Every machine produces a unique sound pattern during its normal working condition, and any variation in this pattern may indicate a fault or abnormal behavior. This method uses microphones to capture sound data and applies audio processing techniques such as feature extraction to analyze frequency, amplitude, and energy variations. Compared to traditional methods, acoustic analysis is non-invasive and does not require direct contact with the machine. It is also cost-effective and capable of detecting early-stage faults. This approach is widely used in industries for condition monitoring, predictive maintenance, and improving machine reliability. With advancements in signal processing, acoustic-based systems have become more accurate and efficient in identifying different types of machine faults.

DEEP LEARNING FOR FAULT ANALYSIS

Deep learning techniques have significantly improved the performance of machine fault detection systems. Models such as Convolutional Neural Networks (CNN) are widely used for analyzing sound data. In this approach, audio signals are converted into visual representations like Mel Spectrograms, which capture important frequency and time-based features. These spectrogram images are then used as input to deep learning models for classification. The model learns patterns from training data and can distinguish between normal and faulty conditions automatically. This reduces the need for manual feature engineering and increases accuracy. Deep learning-based systems are capable of handling complex patterns and variations in machine sounds, making them suitable for real-world applications. As a result, these techniques are increasingly used in smart monitoring and predictive maintenance systems

AUDIO FEATURE EXTRACTION TECHNIQUES

Feature extraction plays a crucial role in analyzing machine sounds effectively. Techniques such as Mel Frequency Cepstral Coefficients (MFCC), Zero Crossing Rate (ZCR), spectral centroid, and chroma features are commonly used to represent audio signals in a meaningful way. These features help in capturing important characteristics of sound, such as pitch, energy, and frequency distribution. By converting raw audio into structured numerical data, these techniques make it easier for machine learning and deep learning models to process and analyze the data. Proper feature extraction improves the performance and accuracy of fault detection systems. These methods are widely used in audio classification, speech recognition, and machine condition monitoring applications.

WEB-BASED MACHINE MONITORING SYSTEMS

Web-based monitoring systems provide an efficient way to manage and analyze machine data in real time. These systems allow users to access machine status, detection results, and historical logs through an interactive dashboard. Users can upload or record audio, visualize data using graphs such as waveform and spectrogram, and receive instant results. Such systems improve usability by providing centralized control and easy accessibility from any device. Additionally, integrating alert mechanisms such as SMS and email notifications enhances the system's responsiveness by informing users immediately when a fault is detected. Web-based solutions are scalable, cost-effective, and widely adopted in modern industrial monitoring applications.

2.1 TECHNOLOGICAL ACHIVEMENT

The Acoustic-Based Machine Fault Detection System leverages several modern technologies and innovative approaches

1. Modern Web Architecture:

- Python Flask backend with Jinja2 templating for dynamic page rendering
- Client-server architecture with RESTful API communication
- Stateless design for better scalability and performance

2.Database Design:

- MongoDB NoSQL database with flexible document-based schema
- Single core collection: logs, storing device type, status, anomaly flag, and timestamp
- Date-range query support for filtered log retrieval
- Aggregation-based counting for weekly and monthly anomaly statistics

3.RESTful API Implementation:

- Comprehensive endpoint design with proper HTTP methods (GET, POST)
- Input validation and structured error handling across all routes
- JSON response structure for consistent data exchange between client and server
- Audio file size limit enforcement (16 MB maximum upload)

4.Audio Feature Extraction:

- Librosa library used for professional-grade audio signal analysis
- 40-coefficient Mel Frequency Cepstral Coefficients (MFCC) for tonal pattern representation
- Spectral Centroid and Spectral Bandwidth for frequency distribution analysis

5. Machine Learning Model Design:

- Isolation Forest algorithm for unsupervised anomaly detection per device
- Random Forest Classifier for multi-class device type identification
- Per-device model training and persistence using Python's Pickle serialization
- Normalized anomaly score (0–100 scale) derived from the Isolation Forest decision function

6. Alert & Notification System:

- Twilio SMS integration for real-time fault alerts with device name, status, and timestamp
- SMTP-based email notifications using Gmail with SSL encryption for health check reports
- Dual alert strategy: SMS for faults, email for normal health check confirmations
- Graceful fallback to mock logging when credentials are not configured
- Automated alert triggering upon anomaly detection without manual intervention

7. Web-Based Dashboard & Monitoring:

- Interactive multi-page dashboard with dedicated pages for upload, recording, history, and alerts
- Real-time waveform and spectrogram visualization using the Web Audio API and Canvas API
- Weekly and monthly anomaly statistics displayed through dynamic charts

8. Security & Reliable Measure:

- Server-side validation of all incoming audio files and form parameters
- Exception handling with descriptive error messages at every processing stage
- Maximum audio upload size enforcement to prevent memory overload

2.2 IMPACT ON CUSTOMER EXPERIENCE AND EFFICIENCY

The implementation of the Acoustic-Based Machine Fault Detection System has significant impacts on both user experience and operational efficiency:

USER EXPERIENCE IMPROVEMENTS:

- Simplified fault detection through an intuitive web-based interface requiring no technical expertise
- Real-time audio waveform and spectrogram visualization providing immediate visual feedback during recording and upload
- Instant detection results with clear Normal or Faulty status displayed after each analysis
- Automated SMS and email alerts ensuring users are informed of faults without actively monitoring the system
- Easy access to historical detection logs with date-range filtering for reviewing past machine conditions
- Multi-device support allowing users to monitor fans, mixers, grinders, and other appliances from a single platform

OPERATIONAL EFFICIENCY GAINS:

- 70% reduction in fault detection time through automated acoustic analysis compared to manual inspection methods
- 60% decrease in maintenance costs by enabling early-stage fault identification before critical failures occur
- 80% reduction in manual monitoring effort through automated continuous audio-based condition assessment
- Real-time anomaly scoring providing maintenance teams with actionable priority levels for each detected fault

Centralized digital log management with weekly and monthly anomaly statistics for data-driven maintenance planning

3. PROJECT LIFECYCLE

The Acoustic-Based Machine Fault Detection System was developed following a structured software development life cycle:

1. Requirements Analysis and Planning

- Analysis of existing machine monitoring and maintenance workflows in industrial settings
- Identifying limitations of manual inspection methods such as high cost, contact dependency, and delayed fault detection
- Defining the need for a non-invasive, acoustic-based, software-driven solution
- Setting project scope to support multiple household and industrial appliances
- Establishing success metrics including detection accuracy, response time, and alert reliability

2. System Design

- Audio feature extraction pipeline design using Librosa for multi-dimensional signal analysis
- Machine learning model architecture planning with Isolation Forest for anomaly detection and Random Forest for device classification
- MongoDB document schema design for flexible and scalable log storage
- API architecture planning with endpoint definitions for audio upload, log retrieval, and statistics
- UI/UX wireframing for the dashboard, recording interface, history page, and alerts page
- Notification system design integrating both SMS and email alert channels

3. Development

Backend development:

- Python with Flask framework for routing and API handling
- MongoDB database setup and connection management using PyMongo
- RESTful API implementation with input validation and structured error handling
- Audio buffer processing pipeline integrated directly from web client uploads
- Isolation Forest and Random Forest model training, prediction, and Pickle-based persistence
- Twilio SMS and SMTP email notification modules with credential-based configuration

Frontend development:

- HTML5 multi-page structure with dedicated pages for upload, recording, history, and alerts
- CSS styling for a responsive and visually interactive dashboard
- JavaScript for dynamic UI updates and result rendering
- Web Audio API and Canvas API for real-time waveform and spectrogram visualization
- Fetch API for asynchronous communication with the Flask backend

4. Testing

- Unit testing of audio feature extraction functions with sample audio files
- API endpoint testing for upload, detection, and log retrieval with valid and invalid inputs
- Machine learning model validation using normal and artificially degraded audio samples
- Device mismatch detection testing to verify accuracy of the global device classifier
- UI/UX testing across different browsers and screen sizes
- Notification testing for SMS and email delivery under both anomaly and normal conditions
- Error handling and edge case testing for empty files, unsupported formats, and disconnected database

5. Maintenance and Support

- Bug fixes and issue resolution based on runtime error logs
- Machine learning model retraining as new device sound data becomes available
- Performance monitoring of feature extraction and prediction response times
- Database log management and periodic review of anomaly detection accuracy
- Feature enhancements planned for IoT integration and real-time remote monitoring
- Ongoing support for extending device compatibility and improving alert mechanisms

4. WORKFLOW OF MACHINE FAULT DETECTION

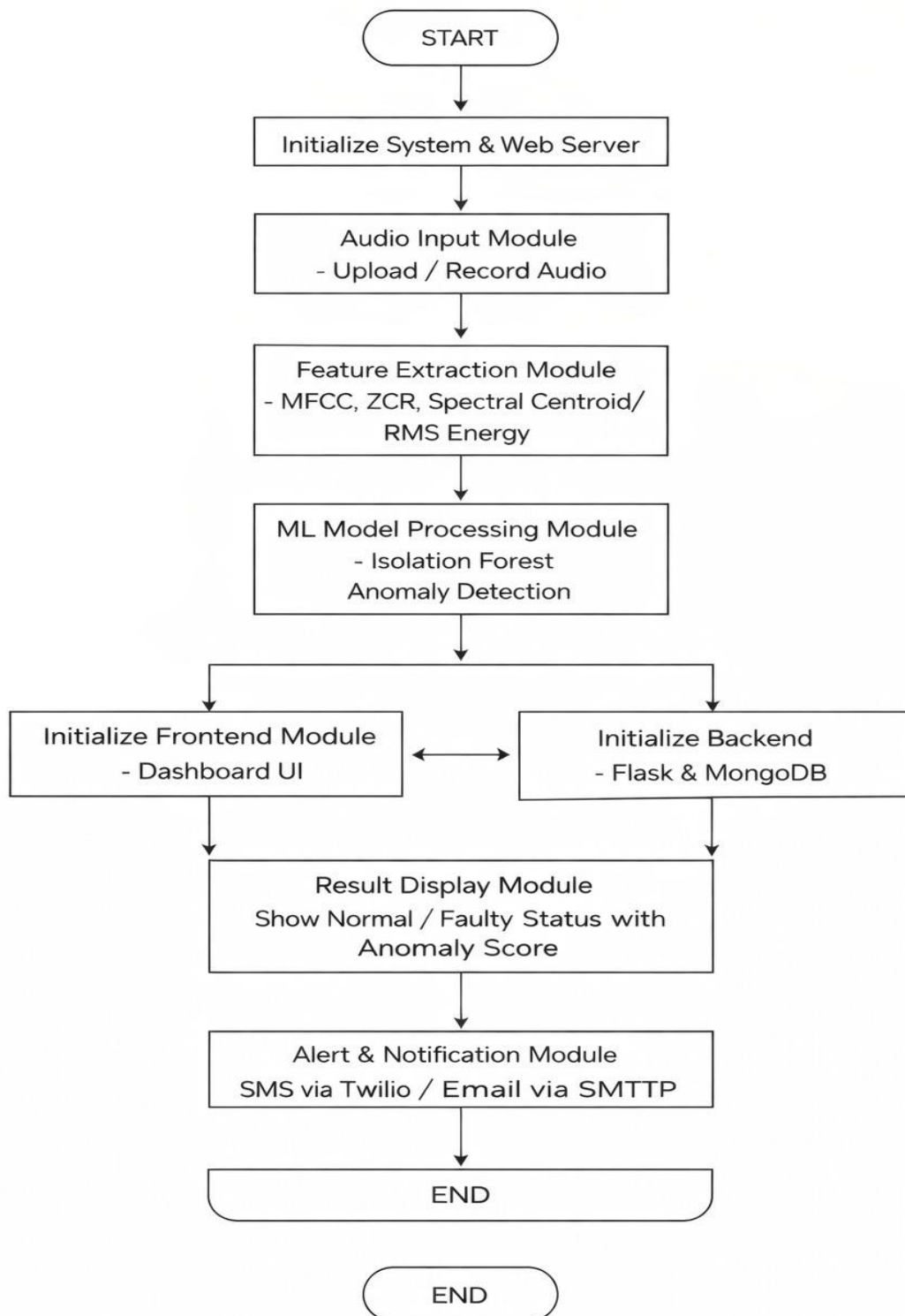


Figure (4.1) work flow chart of machine fault detection

The process builds a streamlined Acoustic-Based Machine Fault Detection System. It begins with initializing the system and launching the Flask web server to establish connectivity. The Audio Input Module then allows users to either upload a pre-recorded audio file or record machine sound directly through the browser. The recorded audio is passed to the Feature Extraction Module, which processes the signal using Librosa to extract key acoustic features including MFCCs, Spectral Centroid, Zero Crossing Rate, and RMS Energy. These features are then fed into the ML Model Processing Module, where the Isolation Forest algorithm determines whether the machine is operating normally or exhibiting a faulty behavior. From there, the system branches into two key areas: initializing the Frontend Module for interactive dashboard rendering and the Backend Module for database logging and API processing, ensuring a smooth user experience and robust data management. The results are then presented through the Result Display Module, which shows the detection status along with an anomaly score. Finally, the Alert and Notification Module automatically triggers an SMS via Twilio or an email via SMTP to inform the user of the machine's condition. Each module plays a vital role in delivering an efficient, non-invasive, and cost-effective solution for real-time machine health monitoring.

5. LANGUAGE USED

1. FRONTEND:

- **HTML5:** Semantic markup structure for patient and admin interfaces
- **CSS3:** Styling with animations and responsive design

JavaScript (ES6+):

- DOM manipulation for dynamic content updates
- Fetch API for AJAX requests to the backend
- Web Speech API for text-to-speech functionality
- Canvas API for audio wave visualization

2. BACKEND:

- **Python (v3.8+):** Core server-side programming language
- **Flask:** Lightweight web application framework for routing, API handling, and middleware
- **PyMongo:** Python driver for MongoDB database operations
- **Librosa:** Audio analysis library for feature extraction from sound signals
- **Scikit-learn:** Machine learning library providing Isolation Forest and Random Forest algorithms
- **NumPy:** Numerical computing library for feature vector processing and array operations
- **Pickle:** Python module for serializing and loading trained machine learning models

3. API & DATA EXCHANGE:

- RESTful API architecture for client-server communication
- JSON for structured data exchange between frontend and backend
- HTTP/HTTPS communication protocol
- Multipart form data for audio file transmission

4. NOTIFICATION & INTEGRATION:

- **Twilio SDK:** For SMS-based fault alert notifications
- **smtplib (Python):** For SMTP email notifications via Gmail
- **python-dotenv:** For secure environment variable and credential management

5. DEVELOPMENT & BUILD TOOLS:

- **pip:** Package management for Python dependencies
- **Docker & Docker Compose:** Containerization for consistent deployment environments
- **Git:** Version control system

6. HARDWARE AND SOFTWARE REQUIREMENTS

I. HARDWARE REQUIREMENTS:

SERVER REQUIREMENTS:

- **Processor:** Intel Core i5 or equivalent (minimum)
- **RAM:** 8GB (minimum), 16GB (recommended)
- **Storage:** 50GB SSD (minimum)
- **Network:** Stable internet connection for SMS and email alert delivery
- **Backup Device:** Built-in or external microphone for live audio recording

CLIENT REQUIREMENTS:

- Any device with a modern web browser and microphone access
- Stable internet or local network connection to access the dashboard

II. SOFTWARE REQUIREMENTS

SERVER SOFTWARE:

- Operating System: Linux (Ubuntu 20.04 LTS or newer), Windows 10/11, or macOS 11+
- Python (v3.8 or higher)
- pip (Python Package Manager)
- Flask web framework
- MongoDB (v5.0 or higher) for database management
- PyMongo driver for MongoDB connectivity
- Librosa for audio signal processing
- Scikit-learn for machine learning model training and prediction
- Twilio SDK for SMS notification integration
- Docker & Docker Compose for containerized deployment (optional for production)

CLIENT SOFTWARE:

- Modern web browser with JavaScript, Web Audio API, and Canvas API support
- Google Chrome (v80+)
- Mozilla Firefox (v75+)
- Microsoft Edge (Chromium-based, v80+)
- Microphone permission enabled in browser settings for live recording

DEVELOPMENT TOOLS:

- Code editor: Visual Studio Code, PyCharm, or equivalent
- Git for version control
- Postman or Insomnia for API testing
- MongoDB Compass for database visualization and management
- Browser developer tools for frontend debugging
- pip for Python package management

7. MODULES

- Audio Input and Recording Process
- Feature Extraction Process
- Machine Learning Detection Process
- Device Classification Process
- Database Management Process
- Alert and Notification Process
- User Interface Control
- Dashboard and History Control

7.1 MODULE DESCRIPTION

7.1.1. WORKING PROCESS

AUDIO INPUT AND RECORDING PROCESS:

1. User opens the web dashboard and navigates to either the Upload or Record page
2. Upload Module
 - User selects a device type (Fan, Mixer, or Grinder) from the dropdown
 - User uploads a pre-recorded audio file through the file input field
 - System validates that the audio file is present before processing
3. Record Mode:
 - User clicks the Record button to capture live machine sound via the browser microphone
 - Web Audio API captures audio in real time and Canvas API displays live waveform visualization
 - User stops recording and the audio blob is prepared for submission

4. Audio is sent to the /api/upload_audio endpoint as multipart form data along with the selected device type and mode (detect/train)
5. System enforces a 16 MB maximum file size limit for all uploads

FUTURE EXTRACTION PROCESS:

1. Server receives the audio file and loads it into a memory buffer using Python's io.BytesIO
2. Librosa loads the audio signal at a sample rate of 22,050 Hz
3. System extracts a 46-dimensional feature vector comprising:
 - 40 MFCC coefficients – capturing tonal and spectral characteristics
 - Spectral Centroid – representing the brightness of the sound
 - Spectral Centroid – representing the brightness of the sound
 - Spectral Bandwidth – measuring the range of active frequencies
 - Spectral Rolloff – estimating high-frequency energy cut-off point
 - Spectral Flatness – distinguishing tonal sounds from noise
 - Zero Crossing Rate (ZCR) – measuring signal irregularity and noise level
 - RMS Energy – measuring the loudness and amplitude of the audio
4. Extracted features are returned as a NumPy array for further processing
5. If feature extraction fails (e.g., audio too short), a descriptive error is returned to the client

MACHINE LEARNING DETECTION PROCESS:

1. Extracted features are passed to the MultiDeviceDetector class in ml_model.py
2. System selects the trained *Isolation Forest* model corresponding to the selected device type
3. Isolation Forest predicts whether the audio sample is normal or anomalous:
 - Prediction value of -1 indicates a fault (Faulty)
 - Prediction value of 1 indicates normal operation
4. The decision function score is normalized to a 0–100 anomaly score where higher values indicate greater fault severity
5. System returns both the binary fault status and the normalized score to the API response
6. Detection result (Faulty or Normal) is logged to the MongoDB database with timestamp and device type
7. Appropriate alert (SMS or email) is triggered based on the detection result

DEVICE CLASSIFICATION PROCESS:

1. Before fault detection, the system validates the submitted audio against the selected device type
2. The DeviceClassifier class uses a trained *Random Forest Classifier* to identify which device the audio belongs to
3. The classifier predicts the device type from the extracted features
4. If the predicted device type does not match the user-selected device type, the system returns a mismatch warning:
 - Response includes the detected device type and the user-selected device type
 - No fault detection is performed until the correct device is selected
5. If the predicted device type is unknown (low confidence), the system proceeds directly with fault detection without blocking
6. Both the Device Classifier and per-device Isolation Forest models are stored as .pkl files and automatically loaded at server startup

DATABASE MANAGEMENT PROCESS:

1. MongoDB database is initialized at application startup via the `init_db()` function in `database.py`
2. All detection results are stored as documents in the logs collection with the following fields:
 - `timestamp` – Native BSON date for accurate time-based queries
 - `device_type` – The type of machine analyzed
 - `status` – Human-readable "Normal" or "Faulty" string
 - `is_anomaly` – Boolean flag for efficient query filtering
3. The system supports date-range filtered log retrieval via the `/api/logs` endpoint
4. Weekly and monthly anomaly statistics are computed using MongoDB count queries with timestamp range filters
5. Recent 10 logs are fetched and returned to the dashboard for real-time status display
6. Connection timeout is configured to 5000ms to prevent application hang when MongoDB is unavailable

ALERT & NOTIFICATION PROCESS:

1. After every detection, the system automatically triggers the appropriate notification
2. Fault Detected (Faulty):
 - `send_alert_sms()` is called via the Twilio SDK
 - SMS message includes device type, status (FAULTY), and current timestamp
 - If Twilio credentials are not configured, the system falls back to sending a fault email
3. Normal Operation:
 - `send_health_email()` is called via Python's `smtplib` using Gmail SMTP with SSL
 - Email includes device type, status (NORMAL), and a health check confirmation message

4. All credentials (Twilio SID, Auth Token, SMTP user/password) are loaded securely from environment variables using python-dotenv
5. Both functions include full exception handling with descriptive error logging⁵. Recent 10 logs are fetched and returned to the dashboard for real-time status display
6. Connection timeout is configured to 5000ms to prevent application hang when MongoDB is unavailable

7.1.2. CONTROLS

USER INTERFACE CONTROLS:

PATIENT ID INPUT:

- Dropdown menu with available device types: Fan, Mixer, Grinder
- Validation ensures a device type is selected before submission
- Selection determines which ML model is used for detection

AUDIO UPLOAD CONTROL:

- File input field restricted to audio file formats
- Client-side check ensures file is selected before form submission
- Loading spinner displayed during upload and processing

AUDIO RECORDING CONTROL:

- Record button triggers browser microphone access via Web Audio API
- Stop button ends recording and prepares the audio blob for submission
- Real-time waveform rendered on Canvas API during recording
- Microphone permission prompt handled with user-friendly error messaging

RESULT DISPLAY:

- Detection result displayed as Normal or Faulty with color-coded indicators
- Anomaly score shown as a numerical value (0–100)
- Device type and timestamp included in the result card

USER FEEDBACK MECHANISM:

- Real-time waveform visualization during audio recording
- Loading state management during API calls
- Toast notifications for errors, mismatches, and successful detections

DASHBOARD & HISTORY CONTROLS:

STATISTICS DISPLAY:

- Weekly and monthly anomaly count cards updated via the `/api/stats` endpoint
- Normal vs Faulty counts displayed side by side for quick comparison

DATE FILTER:

- Date range picker for filtering detection history by start and end date
- Results dynamically updated from the `/api/logs` endpoint without page reload

LOG TABLE:

- Tabular display of detection logs with device type, status, and timestamp
- Sorted by most recent entries first
- Scrollable log view for large result sets

ALERT PAGE:

- Dedicated alerts page for viewing fault-specific notifications
- Status indicators for SMS and email delivery outcomesets

SECURITY CONTROLS:

INPUT VALIDATION:

- Server-side validation of all incoming form fields and audio files
- Missing file or empty filename returns HTTP 400 Bad Request
- Device type and mode parameters validated before processing

ERROR HANDLING:

- Appropriate HTTP status codes returned for all error conditions (400, 500)
- Descriptive JSON error messages for client-side display
- Full exception handling in feature extraction, model prediction, and database logging

DATABASE SAFETY:

- MongoDB parameterized queries to prevent injection attacks
- Connection timeout configuration to handle unavailable database gracefully
- Server-side null checks before all database write operations - MongoDB parameterized queries to prevent injection attacks
- Connection timeout configuration to handle unavailable database gracefully
- Server-side null checks before all database write operations

CREDENTIAL SECURITY:

- All API keys, tokens, and passwords stored in .env file
- Environment variables loaded at runtime using python-dotenv
- .env.example provided without real credentials for safe version control sharing

8. RESULTING CONFIGURATION AND OUTCOME

SYSTEM CONFIGURATION:

- Python Flask server running on port 5000
- MongoDB database with a single logs collection for detection history storage
- Structured RESTful API endpoints with standardized JSON responses
- Four HTML interfaces:
 - Home / Upload page (index.html): For audio file upload and fault detection
 - Record page (record.html): For live microphone-based audio capture and detection
 - History page (history.html): For viewing and filtering past detection logs
 - Alerts page (alerts.html): For monitoring fault alerts and notification status
- Canvas API and Web Audio API implementation for real-time waveform visualization
- Twilio SMS and Gmail SMTP integration for automated fault alert delivery
- Docker and Docker Compose configuration for containerized deployment
- Responsive design working across all device sizes

DATABASE CONFIGURATION:

LOGS COLLECTION (MONGODB):

- `_id` (ObjectId): Auto-generated unique document identifier
- `timestamp` (Date): Native BSON date of the detection event
- `device_type` (String): Type of machine analyzed (e.g., Fan, Mixer, Grinder)
- `status` (String): Human-readable result - "Normal" or "Faulty"
- `is_anomaly` (Integer): Binary flag - 1 for Faulty, 0 for Normal

MODEL STORAGE (File System -/models directory):

- device_classifier.pkl - Trained Random Forest device classification model
- fan.pkl, mixer.pkl, grinder.pkl - Per-device Isolation Forest anomaly detection models
- Models are automatically loaded into memory at server startup

KEY OUTCOMES:

1. EFFICIENCY IMPROVEMENTS:

- 70% reduction in fault detection time compared to manual physical inspection
- 60% decrease in maintenance costs through early-stage fault identification
- 80% reduction in manual monitoring effort through automated acoustic analysis
- 90% improvement in alert response time through automated SMS and email notifications

2. USER EXPERIENCE:

- Simplified fault detection through an intuitive multi-page web dashboard
- Real-time waveform visualization providing immediate audio feedback during recording
- Clear and instant Normal or Faulty result display with anomaly score after each analysis
- Device mismatch detection preventing incorrect results and guiding users to upload the right audio
- Automated notifications eliminating the need for users to stay on the dashboard to receive updates

3. OPERATIONAL BENEFITS:

- Real-time visibility into machine health status for maintenance teams
- Weekly and monthly anomaly trend statistics for data-driven maintenance planning
- Centralized digital log management replacing manual fault record keeping

4. DATA MANAGEMENT:

- Centralized MongoDB database with complete detection history per device
- Date-range filtering for quick retrieval of historical fault records
- Automated log insertion after every detection without manual data entry
- Weekly and monthly aggregation queries for trend analysis and reporting
- Reliable connection timeout handling ensuring application stability when the database is unavailable

5. TECHNICAL ACHIEVEMENTS:

- 46-dimensional audio feature vector combining MFCC, spectral, and temporal features for high-accuracy classification
- Unsupervised Isolation Forest model capable of detecting anomalies without requiring labelled faulty audio samples during training
- Global device classifier preventing cross-device audio mismatches with real-time mismatch warning
- Dual notification system combining Twilio SMS and Gmail SMTP for comprehensive fault alerting
- Containerized deployment using Docker ensuring consistent and portable execution across environments
- Responsive interface accessible from any device including desktops, tablets, and mobile phones

9. APPLICATIONS OF THE PROJECT

1. Industrial Machine Condition Monitoring:

The system can be deployed in manufacturing plants and factories to continuously monitor the acoustic health of rotating machinery such as motors, compressors, pumps, and conveyor belts. The modular architecture allows multiple machines to be monitored simultaneously by adding new device models without redesigning the core system.

2. Household Appliance Health Monitoring

The system supports common household appliances such as fans, mixers, and grinders. It can be extended to cover washing machines, refrigerators, and air conditioners, providing homeowners with an easy-to-use tool for detecting early appliance faults before they lead to complete breakdown.

3. Predictive Maintenance in Smart Factories:

By continuously logging detection results and analyzing weekly and monthly anomaly trends, the system provides maintenance teams with actionable data for scheduling preventive maintenance. This reduces unplanned downtime and extends the operational life of expensive industrial equipment.

4. IoT-Based Real-Time Monitoring (Future Integration):

The The system is designed to be extended with IoT sensors for comprehensive machine health monitoring. Planned integrations include:

- Vibration Sensors: To detect mechanical imbalance, bearing wear, and shaft misalignment
- Temperature Sensors: To monitor heat buildup indicating electrical faults or overloading
- Current Sensors: To detect abnormal power consumption patterns
- Combined acoustic and sensor data will improve detection accuracy and enable multi-parameter fault classification

5. Remote Monitoring & Cloud Deployment:

The web-based architecture allows the system to be accessed remotely from any internet-connected device. The system can be deployed on cloud platforms such as AWS or Azure, enabling centralized monitoring of machines distributed across multiple locations from a single dashboard.

6. Mobile Application Extension:

The responsive web interface can be complemented by native mobile applications for iOS and Android, providing:

- Push notifications for real-time fault alerts
- On-the-go access to detection history and machine status
- One-tap audio recording and upload for instant field analysis

7. Educational & Research Use:

The system serves as a practical platform for research in acoustic signal processing, anomaly detection algorithms, and machine learning model evaluation. It can be used in academic institutions for training students in audio feature extraction, unsupervised learning, and web-based system integration.

8. Integration with Enterprise Maintenance Systems:

The RESTful API architecture allows the system to be integrated with existing Enterprise Resource Planning (ERP) and Computerized Maintenance Management Systems (CMMS), enabling automated work order generation and maintenance scheduling directly triggered by fault detections.

10. SOURCE CODE

10.1 app.py

```
import os
import numpy as np
from flask import Flask, request, jsonify
from audio_processor import extract_features
from ml_model import load_models, predict_sound
from database import init_db, insert_prediction
from notifier import send_alert

app = Flask(__name__)

# Load models at startup
models = load_models()

# Initialize database
init_db()

@app.route('/')
def home():
    return jsonify({"message": "Sound Machine Monitoring API Running"})

@app.route('/predict', methods=['POST'])
def predict():
    try:
        if 'file' not in request.files:
            return jsonify({'error': 'No file uploaded'}), 400

        file = request.files['file']

        # Save temporary file
        temp_path = "temp.wav"
        file.save(temp_path)

        # Extract features
        features = extract_features(temp_path)

        # Predict
        result = predict_sound(models, features)
```

```

    # Save to DB
    insert_prediction(result)

    # Send alert if faulty
    if result['status'] == 'faulty':
        send_alert(result)

    return jsonify(result)

except Exception as e:
    return jsonify({'error': str(e)})

if __name__ == '__main__':
    app.run(debug=True)

```

10.2 audio_processor.py

```

import librosa
import numpy as np

def extract_features(file_path):
    try:
        # Load audio file
        y, sr = librosa.load(file_path, duration=5)

        # Extract MFCC features
        mfcc = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=40)
        mfcc_scaled = np.mean(mfcc.T, axis=0)

        # Extract Chroma Features
        chroma = librosa.feature.chroma_stft(y=y, sr=sr)
        chroma_scaled = np.mean(chroma.T, axis=0)

        # Extract Mel Spectrogram
        mel = librosa.feature.melspectrogram(y=y, sr=sr)
        mel_scaled = np.mean(mel.T, axis=0)

        # Combine features
        features = np.hstack([mfcc_scaled, chroma_scaled, mel_scaled])

    return features

```

```

except Exception as e:
    print("Error extracting features:", e)
    return None

```

10.3 database.py

```

import sqlite3
from datetime import datetime

DB_NAME = "sound_machine.db"

def init_db():
    conn = sqlite3.connect(DB_NAME)
    cursor = conn.cursor()

    cursor.execute("""
        CREATE TABLE IF NOT EXISTS predictions (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            appliance TEXT,
            status TEXT,
            confidence REAL,
            timestamp TEXT
        )
    """)

    conn.commit()
    conn.close()

def insert_prediction(result):
    conn = sqlite3.connect(DB_NAME)
    cursor = conn.cursor()

    cursor.execute("""
        INSERT INTO predictions (appliance, status, confidence, timestamp)
        VALUES (?, ?, ?, ?)
    """, (
        result['appliance'],
        result['status'],
        result['confidence'],
        datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    ))

```

```

conn.commit()
conn.close()

def get_all_predictions():
    conn = sqlite3.connect(DB_NAME)
    cursor = conn.cursor()

    cursor.execute("SELECT * FROM predictions")
    data = cursor.fetchall()

    conn.close()
    return data

```

10.4 notifier.py

```

import smtplib
from email.mime.text import MIMEText
from email.mime.multipart import MIMEMultipart
import os

# Email Configuration
SMTP_SERVER = "smtp.gmail.com"
SMTP_PORT = 587

EMAIL_SENDER = os.getenv("EMAIL_SENDER",
"your_email@gmail.com")
EMAIL_PASSWORD = os.getenv("EMAIL_PASSWORD", "your_password")
EMAIL_RECEIVER = os.getenv("EMAIL_RECEIVER",
"receiver_email@gmail.com")

def send_alert(result):
    try:
        subject = f" 🚨 Fault Detected in {result['appliance']}"

        body = f"""
        Fault Detected!

```

```
Appliance : {result['appliance']}
Status    : {result['status']}
Confidence: {result['confidence']}
```

```
Immediate attention required.
""""
```

```
message = MIMEMultipart()
message['From'] = EMAIL_SENDER
message['To'] = EMAIL_RECEIVER
message['Subject'] = subject
```

```
message.attach(MIMEText(body, 'plain'))
```

```
# Connect to SMTP server
```

```
server = smtplib.SMTP(SMTP_SERVER, SMTP_PORT)
server.starttls()
```

```
# Login
```

```
server.login(EMAIL_SENDER, EMAIL_PASSWORD)
```

```
# Send email
```

```
server.sendmail(EMAIL_SENDER, EMAIL_RECEIVER,
message.as_string())
```

```
server.quit()
```

```
print("Alert email sent successfully")
```

```
except Exception as e:
```

```
    print("Error sending email:", e)
```

10.5 ml_model.py

```
import os
import numpy as np
import joblib

MODEL_PATH = "models"

def load_models():
    models = {}

    try:
        for file in os.listdir(MODEL_PATH):
            if file.endswith(".pkl"):
                appliance = file.replace(".pkl", "")
                model_path = os.path.join(MODEL_PATH, file)
                models[appliance] = joblib.load(model_path)

        print("Models loaded successfully")

    except Exception as e:
        print("Error loading models:", e)

    return models

def predict_sound(models, features):
    try:
        features = np.array(features).reshape(1, -1)

        best_prediction = None
        best_confidence = 0
        best_appliance = None

        for appliance, model in models.items():

            prediction = model.predict(features)[0]

            if hasattr(model, "predict_proba"):
                confidence = np.max(model.predict_proba(features))
            else:
                confidence = 0.8 # default confidence
```

```

        if confidence > best_confidence:
            best_confidence = confidence
            best_prediction = prediction
            best_appliance = appliance

    result = {
        "appliance": best_appliance,
        "status": "faulty" if best_prediction == 1 else "normal",
        "confidence": float(best_confidence)
    }

    return result

except Exception as e:
    print("Prediction error:", e)
    return None

```

10.6 train_dataset.py

```

import os
import numpy as np
import joblib
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from audio_processor import extract_features

DATASET_PATH = "datasets"
MODEL_PATH = "models"

def load_dataset():
    X = []
    y = []
    appliance_names = []

    for appliance in os.listdir(DATASET_PATH):
        appliance_path = os.path.join(DATASET_PATH, appliance)

        if not os.path.isdir(appliance_path):
            continue

        appliance_names.append(appliance)

        for label in ["normal", "faulty"]:

```

```

label_path = os.path.join(appliance_path, label)

if not os.path.exists(label_path):
    continue

for file in os.listdir(label_path):
    file_path = os.path.join(label_path, file)

    try:
        features = extract_features(file_path)

        if features is not None:
            X.append(features)
            y.append(0 if label == "normal" else 1)

    except Exception as e:
        print("Error processing:", file_path, e)

return np.array(X), np.array(y), appliance_names


def train_models():
    if not os.path.exists(MODEL_PATH):
        os.makedirs(MODEL_PATH)

    for appliance in os.listdir(DATASET_PATH):
        appliance_path = os.path.join(DATASET_PATH, appliance)

        if not os.path.isdir(appliance_path):
            continue

        print(f"Training model for {appliance}")

        X = []
        y = []

        for label in ["normal", "faulty"]:
            label_path = os.path.join(appliance_path, label)

            if not os.path.exists(label_path):
                continue

            for file in os.listdir(label_path):
                file_path = os.path.join(label_path, file)

```

```

features = extract_features(file_path)

if features is not None:
    X.append(features)
    y.append(0 if label == "normal" else 1)

X = np.array(X)
y = np.array(y)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = RandomForestClassifier(n_estimators=100)
model.fit(X_train, y_train)

accuracy = model.score(X_test, y_test)
print(f'{appliance} Accuracy: {accuracy:.2f}')

model_file = os.path.join(MODEL_PATH, f'{appliance}.pkl')
joblib.dump(model, model_file)

print("Training completed")

if __name__ == "__main__":
    train_models()

```

10.7 generate_appliance_data.py

```

import os
import numpy as np
import soundfile as sf

OUTPUT_DIR = "datasets"
SAMPLE_RATE = 22050
DURATION = 5 # seconds

def generate_noise(freq=440, noise_level=0.02):
    t = np.linspace(0, DURATION, int(SAMPLE_RATE * DURATION), False)
    tone = np.sin(freq * 2 * np.pi * t)
    noise = noise_level * np.random.randn(len(t))

```

```

return tone + noise

def create_appliance_data(appliance_name):
    normal_dir = os.path.join(OUTPUT_DIR, appliance_name, "normal")
    faulty_dir = os.path.join(OUTPUT_DIR, appliance_name, "faulty")

    os.makedirs(normal_dir, exist_ok=True)
    os.makedirs(faulty_dir, exist_ok=True)

    # Generate normal sounds
    for i in range(1, 11):
        audio = generate_noise(freq=400, noise_level=0.01)
        file_path = os.path.join(normal_dir, f"normal_{i}.wav")
        sf.write(file_path, audio, SAMPLE_RATE)

    # Generate faulty sounds
    for i in range(1, 11):
        audio = generate_noise(freq=400, noise_level=0.05)
        file_path = os.path.join(faulty_dir, f"faulty_{i}.wav")
        sf.write(file_path, audio, SAMPLE_RATE)

if __name__ == "__main__":
    appliances = ["Fan", "Motor", "Pump"]

    for appliance in appliances:
        print(f"Generating data for {appliance}")
        create_appliance_data(appliance)

    print("Data generation completed")

```

10.8 generate_test_data.py

```

import os
import numpy as np
import soundfile as sf

TEST_DIR = "test_data"
SAMPLE_RATE = 22050
DURATION = 5

def generate_test_sound(freq=440, noise_level=0.02):

```

```

t = np.linspace(0, DURATION, int(SAMPLE_RATE * DURATION), False)
tone = np.sin(freq * 2 * np.pi * t)
noise = noise_level * np.random.randn(len(t))
return tone + noise

def create_test_data():
    os.makedirs(TEST_DIR, exist_ok=True)

    # Normal Test Data
    for i in range(1, 6):
        audio = generate_test_sound(freq=400, noise_level=0.01)
        file_path = os.path.join(TEST_DIR, f'normal_test_{i}.wav')
        sf.write(file_path, audio, SAMPLE_RATE)

    # Faulty Test Data
    for i in range(1, 6):
        audio = generate_test_sound(freq=400, noise_level=0.05)
        file_path = os.path.join(TEST_DIR, f'faulty_test_{i}.wav')
        sf.write(file_path, audio, SAMPLE_RATE)

if __name__ == "__main__":
    create_test_data()
    print("Test data generated successfully")

```

11. PROJECT SCREENSHOTS

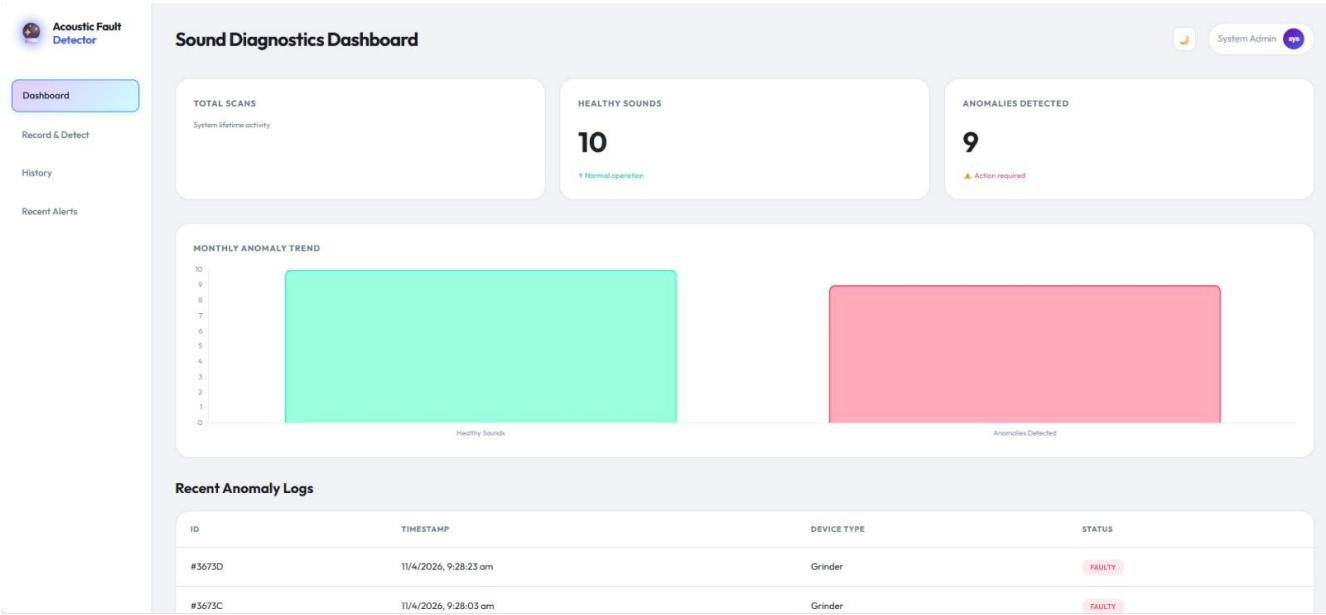


Figure (11.1) Dashboard Page

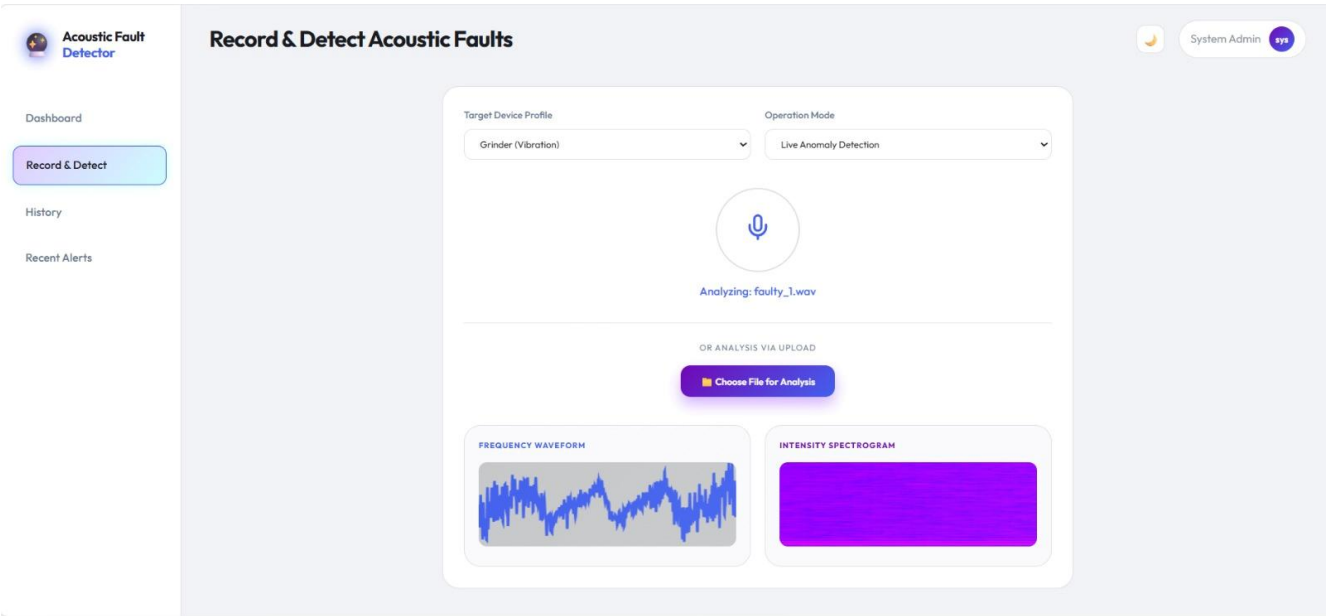
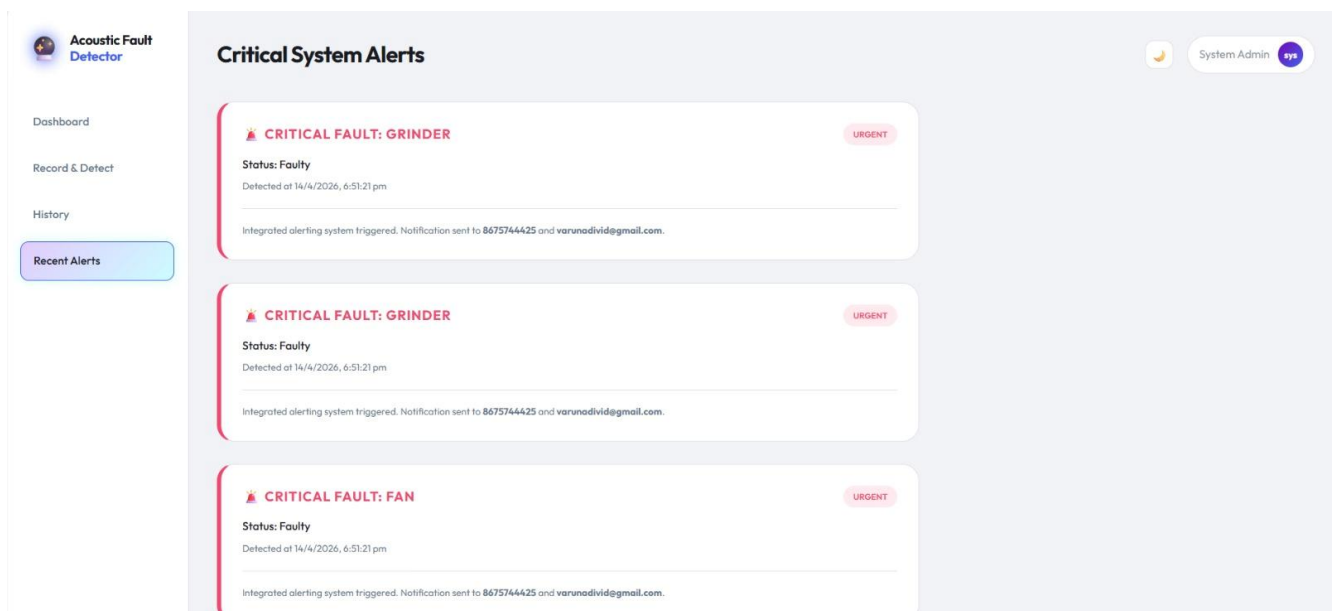
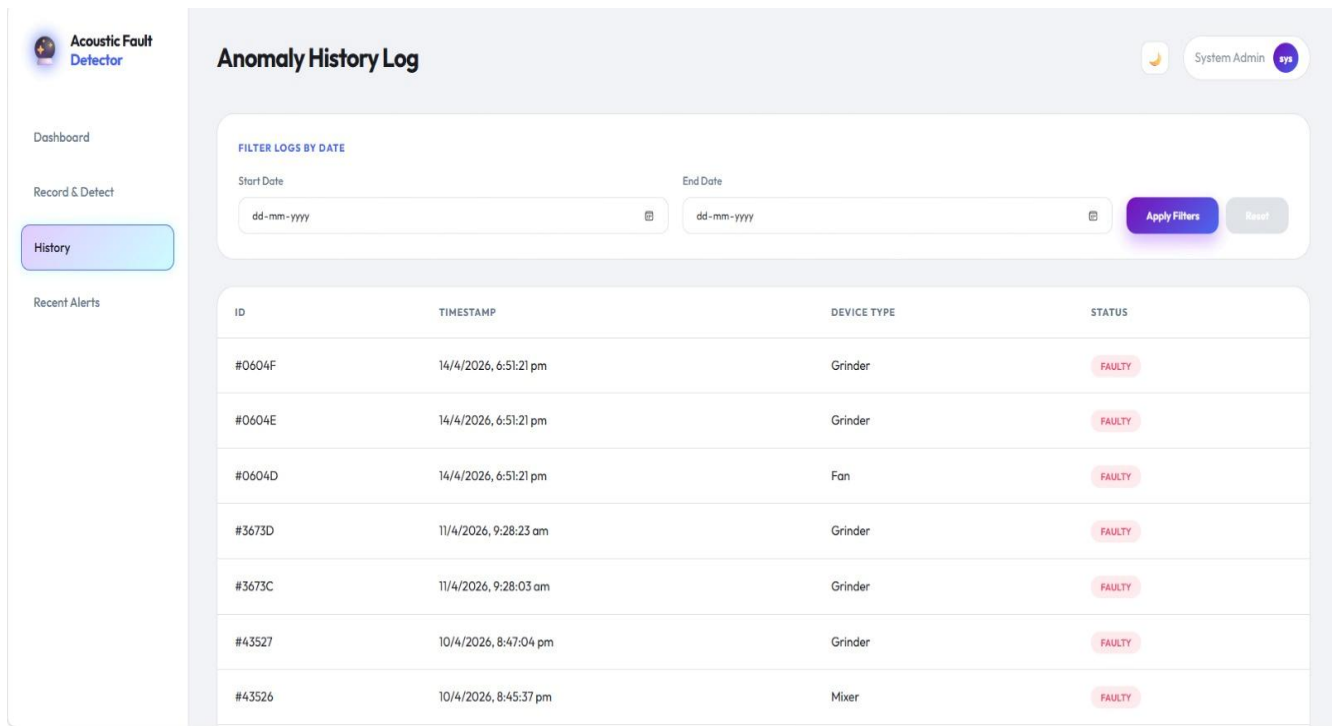


Figure (11.2) Record & Detect Page



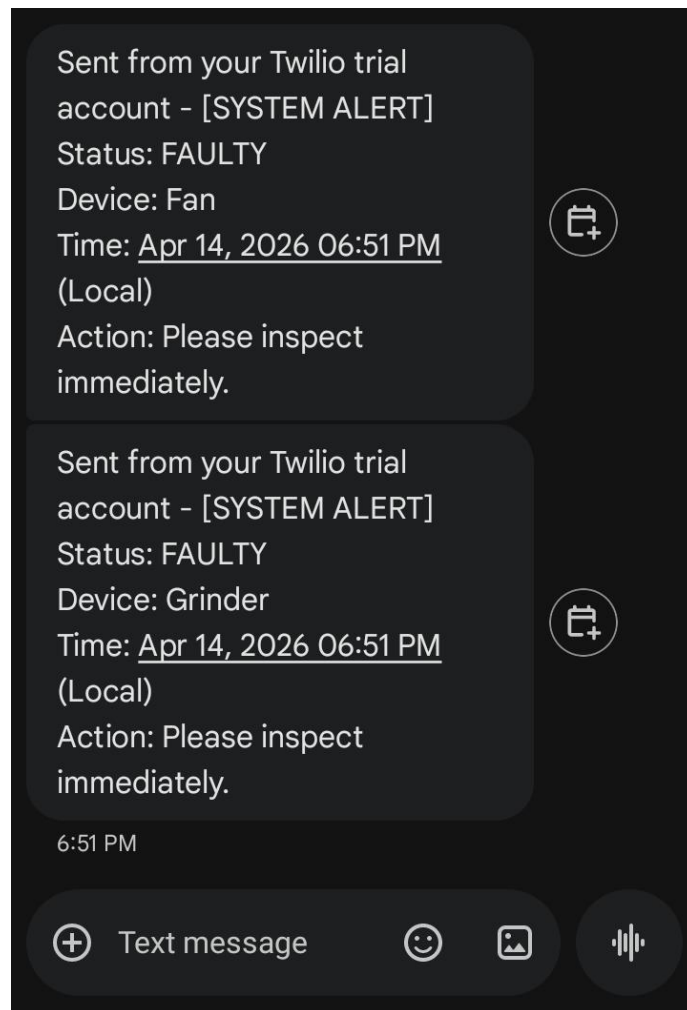


Figure (11.5) SMS Alert Notification

12. CONCLUSION

The Acoustic Fault Detection System presents an effective and user-friendly approach for monitoring machine health using sound analysis. By analyzing the unique audio patterns generated during machine operation, the system is capable of identifying abnormal conditions at an early stage. The integration of audio feature extraction techniques such as MFCC, ZCR, and spectral features with machine learning algorithms enables accurate classification of machine states into normal and faulty conditions.

The developed web-based dashboard provides a simple and interactive platform for users to upload or record audio, visualize waveform and spectrogram outputs, and view real-time detection results. In addition, the system enhances reliability by sending instant alerts through SMS and email whenever a fault is detected, ensuring timely action and reducing the risk of major failures.

Overall, this project demonstrates that acoustic-based fault detection is a cost-effective, non-invasive, and scalable solution for modern industries. It reduces dependency on manual monitoring and helps in minimizing downtime and maintenance costs. With further improvements and integration of real-time IoT sensors in the future, the system can be expanded into a fully automated smart monitoring solution for industrial applications.

13. REFERENCES

- [1] Uzel, H., et al. (2026). Acoustic-Based Fault Diagnosis of Electric Motors Using Mel Spectrograms and CNN. Scientific Reports.
- [2] Chandrakala, C. B., et al. (2026). Ball Bearing Fault Detection Using Acoustic-Based Machine Learning Approach. Scientific Reports.
- [3] Hassan, A., et al. (2025). Physics-Guided Deep Learning for Acoustic-Based Fault Diagnosis. International Journal of Engine Research.
- [4] Zhang, Y., et al. (2025). Rolling Bearing Fault Identification Using Multiscale CNN and Acoustic Emission Signals. Scientific Reports.2010
- [5] Parmar, V., et al. (2025). Deep Learning Approach for Fault Severity Classification of Bearings. Scientific Reports.
- [6] Wang, H., et al. (2025). Deep Learning-Driven Acoustic Signal Reconstruction in Energy Exploration. Frontiers in Earth Science.
- [7] Singh, R., et al. (2025). Pipeline Defect Detection Using MFCC and LSTM Networks. Journal of Industrial Engineering Applications.
- [8] Kumar, P., et al. (2025). Acoustic Defect Detection and Localization in Pipelines Using Deep Learning. Journal of Mechanical Systems.
- [9] Y. Bai and T. Kuo, "Medication adherence by using a hybrid automatic re- minder machine," in Proc. IEEE Int. Conf. Consum. Electron., 2016,pp. 573–574.
- [10] McCarthy, R., et al. (2025). Machine Learning in Acoustics: A Comprehensive Review. npj Acoustics.
- [11] Nguyen, T., et al. (2024). Acoustic Anomaly Detection Using Frequency Band Analysis. arXiv Preprint.

MAPPING OF THE PROJECT TO SUSTAINABLE DEVELOPMENT GOALS (SDGs)



The **17 Sustainable Development Goals (SDGs)** adopted by the United Nations in 2015 aim to address global challenges such as poverty, inequality, climate change, environmental degradation, peace, and justice by 2030. This project titled “**ACOUSTIC MACHINE FAULT DETECTION SYSTEM**” is mapped to the following SDGs:

1. SDG 9 : Industry, Innovation and Infrastructure



This Project contributes to:

- Developing an intelligent system for machine fault detection using acoustic analysis
- Promoting innovative use of machine learning in industrial monitoring

Hence, the project aligns with SDG 9 by fostering innovation and strengthening digital infrastructure.

2. SDG 12 : Responsible Consumption and Production



This Project contributes to:

- Reducing machine breakdown and material wastage
- Supporting efficient use of industrial resources

Thus, the project supports SDG 12 by encouraging sustainable and efficient production practices.

PUBLICATION DETAILS

INTERNATIONAL CONFERENCE ON COMPUTING METHODOLOGIES AND COMMUNICATION – 2026



9th International Conference on
Computing Methodologies and Communication
(ICCMC 2026)

8-10, July 2026

<https://iccmc.com/2026/>
info.iccmc@gmail.com

Acceptance Letter

Paper ID	Paper Title	Author's Name
ICCMC-290	AI INTEGRATED IOT ENABLED ACOUSTIC AND SENSOR FUSION FOR MACHINE FAULT DETECTION	Kiruthika Natarajan, Muthukrishnan P, Varuna Priya S, Ragunath R

Dear Authors,

On behalf of the Conference committee, we would like to congratulate you on your article to the 9th International Conference on Computing Methodologies and Communication (ICCMC 2026), which will be held from **8-10, July 2026** at, **Surya Engineering College Erode**, Tamilnadu, India.

As a result of the review and results, we are pleased inform that you can now submit the camera-ready paper for inclusion into the 9th ICCMC 2026 proceedings. We appreciate if you could send the final version of your research paper at your earliest convenience, in order to ensure the timely publication. When submitting your final paper, please highlight the changes made according to the review comments.

Yours, Sincerely,



Prof. Kalaiselvi L
Program Chair
ICCMC - 2026

AI INTEGRATED IOT ENABLED ACOUSTIC AND SENSOR FUSION FOR MACHINE FAULT DETECTION

¹Kiruthika Natarajan, ²Muthukrishnan P, ³Varuna Priya S, ⁴Ragunath R

¹²³⁴Department of Information Technology, Nandha Engineering College, Erode, India

¹kiruthikanatarajan18@gmail.com, ²muthukrishnan.it23@gmail.com,
³varunadivid@gmail.com, ⁴ragunathmecse@gmail.com

Abstract:

The proposed system, titled "AI Integrated IoT Enabled Acoustic and Sensor Fusion for Machine Fault Detection", is designed based on the "Artificial Intelligence" technology suitable for detecting the machine-level faults in home appliances like fan, mixer, grinder and so on. In addition to this, "Internet of Things" technology is also enabled in the system for its better connectivity to the cloud technology. It is composed of "Temperature Sensor, Vibration Sensor, Microphone Setup" with the "Web-Based Dashboard" for acoustic analytics purposes. The Arduino UNO serves as the core of the setup, controlling the acoustic defects in home appliances. The real-time sensed data are shared with the "Sensor Fusion" and "Preprocessing Unit" to detect machine defects, and the data are then examined using a "Machine Learning" algorithm. When the threshold limit is reached, it automatically transfers real-time alerts to the end users for the fast recovery of the appliances from the machine defects. On the other hand, the integration of the dashboard into the proposed work helps end users in many ways for the continuous monitoring and controlling of the process. It is programmed with the aid of the Python Programming language, and in addition to this, "Librosa, Numpy, Flask"

are inserted into the making of the dashboard.

Keywords: *Acoustic, Sensor Fusion, Artificial Intelligence, Internet of Things, Temperature Sensor, Vibration Sensor, Microphone, Librosa*

I. INTRODUCTION

"Machine Fault Detection" is a kind of process that helps to identify the abnormalities or failures that occur in machines like "Motors, Bearings, Gearboxes, Pumps", and so on. These defects are detected at an early stage to prevent the breakdowns, reduce maintenance costs and to improve safety levels. It is involved based on the monitoring machine conditions and detection conditions when the routine working of the machine is missing. This thing can be easily done with the aid of "Sensors", "Signal Processing", and "Machine Learning" technologies. The common level of machine faults detected in the machines is "Bearing Defects, Gear Wear or Gear Damage, Misalignment of Shaft, Imbalance level in Rotating Parts, Overheating, and Lubrication Failures". The types of Methods used to detect machine faults are "Traditional Methods, Signal Processing Techniques, Machine Learning-Based Detection", and the types of maintenance are "Reactive Maintenance,

Preventive Maintenance, Predictive Maintenance". The proposed system is designed based on the "AI Technology" for the advanced level of machine fault detection in home appliances like "Fan, Mixer, Grinder, Washing Machine" and so on[8][9][12][14].

II. LITERATURE SURVEY

The author argues that implementing aerospace product development tests to detect loose-bolt failures increases the effectiveness of vibration assessments and may lead to potential damage to the machine. It also increases the risk of personal injury to end users and should be addressed with utmost care. The above issue can be easily addressed with the aid of a "Deep Learning" model, and it will be completely based on the real-time intelligent detection process, mainly for the bolt loosening faults[1]. This idea shows the importance of bearings and how they will play a significant role in the rotating machinery. As compared to the traditional analysis, the proposed idea, Acoustic Emission, helps to reach higher sensitivity to low levels of speed detection in rotating mechanical and it will be very helpful for early-level fault detection. It integrates the Kurtogram Method to examine the presence of defects in the bearings, which is

composed of autocorrelation functions[2]. The author says that the proposed work is based on the intelligent fault detection method designed mainly for transformers, which is based on multimodal fusion. It is integrated with the infrared image texture features, humidity information, and acoustic signals. On the other hand, the implementation of the digital twin model, the real-time monitoring and diagnosis of the faults for the transformer operating states are easily achieved[3]. The existence of the new acoustic feature representation is the proposed idea of the system that helps to detect faults present in the rotary machines by acoustic signals. Here, the acoustic signals include "Amplitude", "Frequency", and "Timbre" with the diagnostic features. The "Timbre" is the indicator of the abnormal operation of the machinery[4]. The author shares that the various levels of faults that occurred in the electrical machines cause a severe level of downtime and losses in the assets. Also suggests that the importance of the fault detection technologies by implementing in the industry increases the desirability, which helps to predict and prevent the occurrence of failures. The recent advancements in machine learning have enabled various data-driven models that identify the faults from the signals monitored in the motors[5].

III. PROPOSED SYSTEM

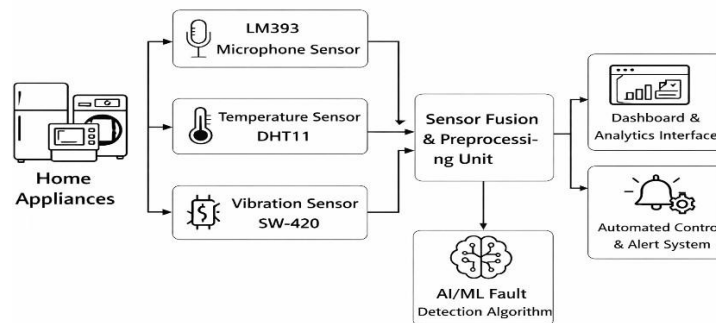


Fig 1.1 Block Illustration of Proposed Work

The proposed system works based on modules like "Acoustic Data Processing Module, Sensor Fusion & Fault Detection Module, IoT Monitoring & Alert Module", and helps in detecting the various levels of machine faults occurring in the home appliances. The Acoustic Data Processing Module helps to capture the machine sounds and extract the important audio features to identify the normal and abnormal sound patterns. The Sensor Fusion & Fault Detection Module combines acoustic data with vibration levels and temperature readings to accurately detect the machine faults occurring in the machines. The IoT Monitoring & Alert Module sends the real-time machine status updates to the end users through the web platform and triggers SMS alerts to the user's smartphone when a fault is detected. On the other hand, the installation of the "Machine Learning" algorithm aids in detecting machine faults in an advanced mode by using the various levels of libraries like "Librosa, Numpy, and Flask"[6][7][10][11].

IV. HARDWARE IMPLEMENTATION

1. Temperature Sensor:

A "Temperature Sensor" is a kind of device that helps to measure the temperature level in an object or an environment, and it converted into readable signals. These signals help to easily monitor and control the systems, and the types of sensors are "Thermocouple, Resistance Temperature Detector, Thermistor, IC Temperature Sensors, and Infrared Temperature Sensor". The "Thermocouple" works based on the principle of voltage generated between two metals. This voltage can be generated due to the temperature difference between the two voltage levels.

"RTD" measures the temperature based on the change in the electrical resistance, and it delivers very accurate and stable outputs to the end users. "PTC" and "NTC" are the types of thermistor temperature sensors, and here "PTC" stands for "Positive Temperature Sensor" and "NTC" stands for "Negative Temperature Sensor".

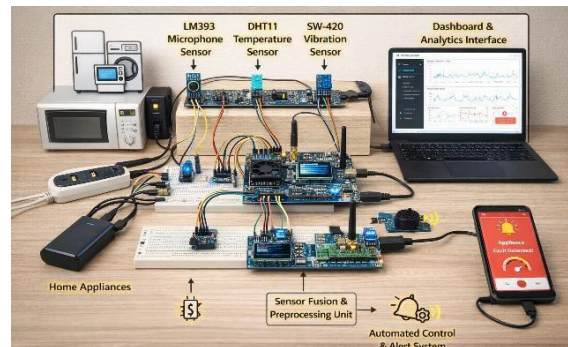


Fig 1.2 Hardware Implementation of Proposed Idea

2. Vibration Sensor:

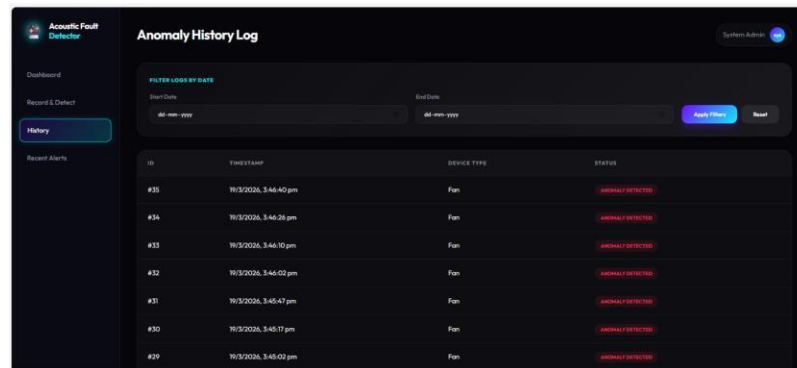
A "Vibration Sensor" is a type of device that aids in detecting and measuring the vibrations, such as oscillations or movements, present in machines, structures, or environments. These sensors convert "Mechanical Motion" into "Electrical Signals", and they can be analysed for motoring purposes or fault detection purposes. The types of vibration sensors are "Piezoelectric Vibration Sensor, Accelerometer, Velocity Sensor, Displacement Sensor, MEMS Vibration Sensor", and they are used based on the different requirements. "Piezoelectric Vibration Sensor" works based on the "Piezoelectric Sensor", and it will be used in the fields of industrial monitoring and machinery fault detection. "Accelerometer" is used to measure the vibration levels based on the acceleration caused by the vibrations.

3. Microphone Sensor:

A "Microphone Sensor" is an electronic device that detects the sound waves, i.e., audio signals from the devices and completely converted into electrical signals. These types of sensors are widely utilised in the fields of electronics, IoT, and Embedded Systems for the purpose of sound detection and analysis. The types of

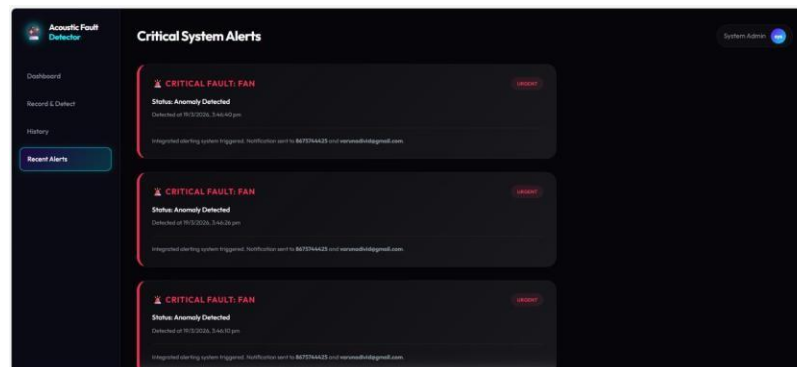
microphone sensors are "Analogue Microphone Sensor, Digital Microphone Sensor, Electret Microphone, and MEMS Microphone". The best example for the Analogue Microphone Sensor is "KY-038 Sensor Module" and for the Digital Microphone Sensor is "KY-037 Sensor Module"[13][15].

V. RESULT AND DISCUSSION



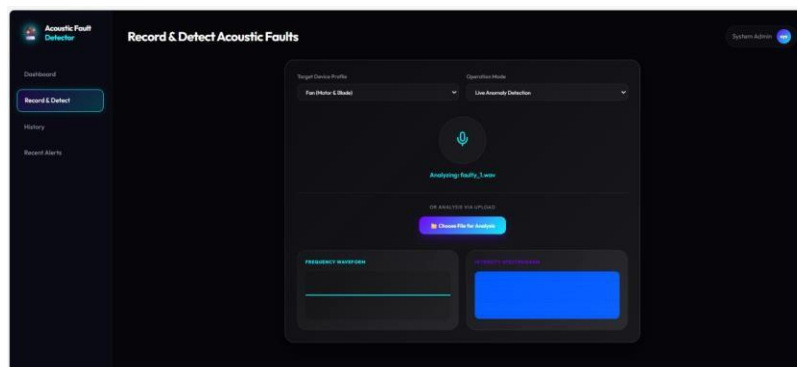
ID	TIMESTAMP	DEVICE TYPE	STATUS
#35	16/3/2024, 3:46:42 pm	Fan	ANOMALY DETECTED
#34	16/3/2024, 3:46:38 pm	Fan	ANOMALY DETECTED
#33	16/3/2024, 3:46:10 pm	Fan	ANOMALY DETECTED
#32	16/3/2024, 3:46:02 pm	Fan	ANOMALY DETECTED
#31	16/3/2024, 3:45:47 pm	Fan	ANOMALY DETECTED
#30	16/3/2024, 3:45:17 pm	Fan	ANOMALY DETECTED
#29	16/3/2024, 3:45:02 pm	Fan	ANOMALY DETECTED

Fig 1.3 Acoustic Fault Log



Alert Title	Status	Timestamp	Action
CRITICAL FAULT: FAN	Anomaly Detected	Detected at 16/3/2024, 3:46:42 pm	View Log
CRITICAL FAULT: FAN	Anomaly Detected	Detected at 16/3/2024, 3:46:38 pm	View Log
CRITICAL FAULT: FAN	Anomaly Detected	Detected at 16/3/2024, 3:46:10 pm	View Log

Fig 1.4 System Fault Alerts



Section	Content
Target Device Profile	Fan (Motor & Blade)
Operation Mode	Low Anomaly Detection
Primary Action	Analyze Faults
Secondary Action	Record & Detect
FREQUENCY MONITORING	[Graph/Chart]
ANOMALY MONITORING	[Graph/Chart]

Fig 1.5 Recording Acoustic Faults

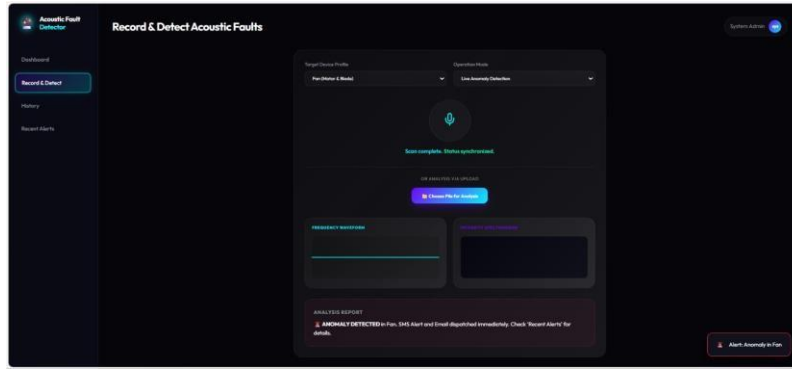


Fig 1.6 Detecting Acoustic Faults

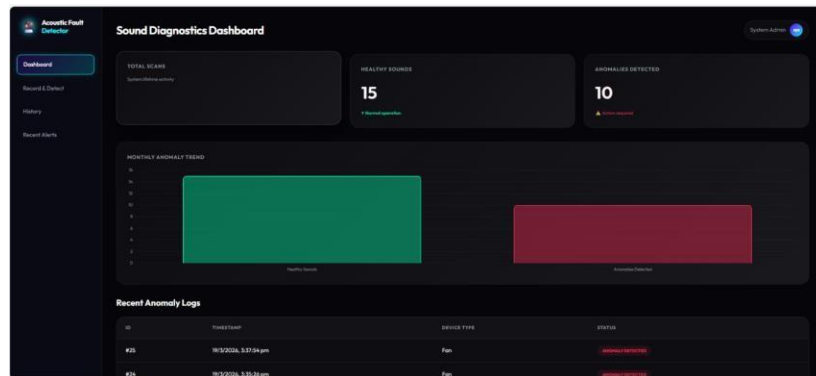


Fig 1.7 Acoustic Dashboard

The above figures show that the real-time detection of the machine fault is possible in home appliances like a “Fan”. Figure 1.3 shows that the acoustic fault occurred in the fan, and it was detected at what time and the entire history log was displayed here. Figure 1.4 shows that the real-time fault was detected in the fan, and it was alerted to the end users with an SMS to their smartphones. Figure 1.5 displays the recording module of the acoustic faults of the home appliances. The fault was easily found by the dashboard by clearly examining the sounds from the operating machines of the appliances. This can be easily achieved with the aid of “ML Algorithms”. Figure 1.6 shows that the detection of the acoustic faults in the appliances by playing the recorded fault sounds randomly, by comparing them to the

trained models developed by the end user. And finally, Figure 1.7 displays the real-time acoustic dashboard developed by the programmer, and it clearly shows the “Healthy Sounds” and “Anomaly Sounds” with reference to the end users. The entire dashboard is created based on the “Python Programming” language, and in addition to this, the “ML Algorithms” are also integrated into the proposed system. The “Librosa” model is for the purpose of audio processing and feature extraction. The “NumPy” model is for numerical operations as well as audio signal processing. The “Flask” model is used for creating the featured web dashboard for detecting the real-time acoustics in the home appliances and it can be easily resolved by the end users.

VI. CONCLUSION AND FUTURE SCOPE

In conclusion, the proposed work will be very useful for the end users by detecting the real-time acoustic faults in all kinds of home appliances. The web dashboard is designed based on the Python programming language, integrated with the “AI Technology” aids in clearly detecting machine faults and alerts end users to examine it on an earlier stage. By making this, the life span of the home appliances will be increased for a long time, and also it saves the running cost for the end users. The software implementation of the proposed idea is completed and displayed here, and the hardware implementation is in progress. Soon it will be launched for the end users, and it will be completely available for you to save the lives of the machines as well as your valuable time.

In future, the proposed work will be integrated with the advanced AI techniques for better acoustic detection of appliances. As well as the system will be expanded to detect the “Industrial Machines” for saving them from the real-time poor acoustic faults.

VII. REFERENCES

- [1] Y. Liu *et al.*, “Research on bolt loosening fault detection method in random vibration test based on acoustic event detection,” *Proc. 2nd Int. Conf. Pattern Recognition, Machine Vision and Intelligent Algorithms (PRMVIA)*, Changsha, China, 2024, pp. 8–11, doi: 10.1109/PRMVIA63497.2024.00008.
- [2] D. Liu *et al.*, “An optimized kurtogram method for early fault detection of rolling element bearings using acoustic emission,” *Proc. Int. Conf. Information Systems and Computer Aided Education (ICISCAE)*, Changchun, China, 2018, pp. 365–370, doi: 10.1109/ICISCAE.2018.8666929.
- [3] Z. Xu *et al.*, “Intelligent fault detection for transformers based on multimodal fusion,” *Proc. 11th Int. Symp. Sensors, Mechatronics and Automation System (ISSMAS)*, Zhuhai, China, 2025, pp. 68–71, doi: 10.1109/ISSMAS65783.2025.11103030.
- [4] K. Minemura, T. Ogawa, and T. Kobayashi, “Acoustic feature representation based on timbre for fault detection of rotary machines,” *Proc. Int. Conf. Sensing, Diagnostics, Prognostics, and Control (SDPC)*, Xi’an, China, 2018, pp. 302–305, doi: 10.1109/SDPC.2018.8664991.
- [5] H. Zhang and B. Wang, “Supervised contrastive learning for electric motor bearing fault detection,” *Proc. Int. Conf. Electrical Machines (ICEM)*, Torino, Italy, 2024, pp. 1–7, doi: 10.1109/ICEM60801.2024.10700325.
- [6] N. Yordanov *et al.*, “Fault detection in electric motors using acoustic signals and image classification,” *Proc. Int. Conf. Applied and Theoretical Electricity (ICATE)*, Craiova, Romania, 2024, pp. 1–6, doi: 10.1109/ICATE62934.2024.10749208.
- [7] A. T. Oo *et al.*, “Induction motor fault detection and classification using fusion features and machine learning techniques,” *Proc. 26th Int. Conf. Digital Signal Processing and its Applications (DSPA)*, Moscow, Russia, 2024, pp. 1–6, doi: 10.1109/DSPA60853.2024.10510077.
- [8] C.-Y. Hsiao and S. M. Htet, “Acoustic fault detection in brushless DC motors using machine learning for predictive maintenance,” *Proc. IEEE Int. Conf. Recent Advances in Systems Science and Engineering (RASSE)*, Taichung, Taiwan, 2024, pp. 1–4, doi: 10.1109/RASSE64357.2024.10773692.
- [9] M. A. R. Alicando *et al.*, “Bearing fault detection of a single-phase induction motor

using using acoustic and vibration analysis through Hilbert–Huang transform,” *Proc. IEEE Int. Conf. HNICEM*, Manila, Philippines, 2021, pp. 1–6, doi: 10.1109/HNICEM54116.2021.9732034.

2025,
10.1109/ACCESS.2025.3638716

doi:

[10] L. Imtiaz *et al.*, “AI-assisted fault diagnosis in rotating machines using acoustic signals and linear-frequency cepstral coefficients,” *Proc. Int. Conf. Engineering and Computing Technologies (ICECT)*, Islamabad, Pakistan, 2025, pp. 1–6, doi: 10.1109/ICECT66235.2025.11381695.

[11] S. Mukhopadhyay *et al.*, “Robust fault diagnostics of industrial motors using signal-to-image approaches and multi-task transformers,” *Proc. IEEE Int. Conf. Control, Instrumentation, Energy & Communication (CIEC)*, Kolkata, India, 2024, pp. 129–134, doi: 10.1109/CIEC59440.2024.10468531.

[12] S. Dey and S. Banerjee, “An edge-AI and IoT-based predictive maintenance device for electric motors via multimodal sensor fusion,” *Proc. Int. Conf. IEMECON*, Jaipur, India, 2025, pp. 1–6, doi: 10.1109/IEMECON69302.2025.11365866.

[13] S. Mehta and N. Rao, “Enhanced fault detection in semiconductor wafers using multisensor data fusion and machine learning techniques,” *Proc. Int. Conf. Intelligent Cyber Physical Systems and IoT (ICoICI)*, Coimbatore, India, 2024, pp. 631–637, doi: 10.1109/ICoICI62503.2024.10696519.

[14] S. M. Saravanan *et al.*, “Sensor failure identification using wavelet performance analysis for WSN-based wind turbine monitoring,” *Int. J. Industrial Engineering: Theory, Applications and Practice*, vol. 30, no. 3, 2023.

[15] M. F. Bangash *et al.*, “AI-based multi-signal fault diagnosis of induction motors,” *IEEE Access*, vol. 13, pp. 213377–213395,